Lab 7

16-bit Barrel Shifter / Rotator & 4-Digit 7-Segment Display

By: Ben Robles & Robert Stevenson
Group U

ECE-3300L
Summer 2026

Date: 07/30/2026

Design:

clock_divider_fixed.v:

```
module clock_divider_fixed(
                        input wire clk_100MHz,
                        output reg clk_2Hz,
                        output reg clk_1kHz
                    );

   reg [15:0] counter_1kHz;
   reg [7:0] counter_2Hz;

   initial begin
      counter_2Hz = 0;
      counter_1kHz = 0;
      clk_2Hz = 0;
      clk_1kHz = 0;
   end

   always @(posedge clk_100MHz) begin
      if(counter_1kHz == 16'd49_999) begin      // 100MHz / (2 x 50000) = 1kHz
         counter_1kHz <= 0;
         clk_1kHz <= ~clk_1kHz;
      end
      else counter_1kHz <= counter_1kHz + 1'b1;
   end

   always @(posedge clk_1kHz) begin
      if(counter_2Hz == 8'd249) begin          //  1kHz / (2 * 250) = 2Hz
         counter_2Hz <= 0;
         clk_2Hz <= ~clk_2Hz;
      end
      else counter_2Hz <= counter_2Hz + 1'b1;
   end

endmodule
```

Description:

This module takes a 100MHz clock and outputs a 1kHz and 2 Hz clock. This is done by a counter and the formula fout = fin / 2*counter. The first counter to create the 1 kHz clock increments to 50k (100MHz / (2 x 50000) = 1kHz) while the 2 Hz clock cascades off the 1 kHz clock at 4 times the frequency (1kHz / (2 * 250) = 2Hz)

debounce_toggle.v:

```
module debounce_toggle(
                        input wire btn_raw,
                        input wire clk_1kHz,
                        input wire rst_n,
```

```
                    output reg btn_toggle
    );
    // Debounce
    reg [2:0] btn_val;
    reg btn_clean;
    reg btn_prev;

    always @(posedge clk_1kHz or negedge rst_n) begin
        if(!rst_n) begin
            btn_val = 3'b000;
            btn_clean = 1'b0;
        end
        else begin
            btn_val = {btn_val[1:0], btn_raw};
            if(btn_val == 3'b111) btn_clean = 1;
            else if(btn_val == 3'b000) btn_clean = 0;
        end
    end

    // Toggle
    always @(posedge clk_1kHz or negedge rst_n) begin
        if (!rst_n) begin
            btn_toggle <= 1'b0;
            btn_prev <= 1'b0;
        end
        else begin
            if(btn_clean && !btn_prev)
                btn_toggle <= ~btn_toggle;

            btn_prev <= btn_clean;
        end
    end

endmodule
```

Description:

This module combines the debounce module and toggle module from previous labs. Debounce is achieved by verifying a steady 3-bit input and toggle acts as a latch.

barrel_shifter16.v:

```
module barrel_shifter16(
                    input [15:0] data_in,
                    input [3:0] shamt,
                    input dir,
                    input rotate,
                    output wire [15:0] data_out
                );

    reg [15:0] stage1_in;
    reg [15:0] stage2_in;
    reg [15:0] stage3_in;
    reg [15:0] stage4_in;
```

```
wire [15:0] stage0;
wire [15:0] stage1_out;
wire [15:0] stage2_out;
wire [15:0] stage3_out;
wire [15:0] stage4_out;

genvar i;

assign stage0 = data_in;

always @(*)
begin
   case ({rotate, dir})
      2'b00: begin    // Shift left
         stage1_in = {stage0[14:0], 1'b0};
         stage2_in = {stage1_out[13:0], 2'b00};
         stage3_in = {stage2_out[11:0], 4'b0000};
         stage4_in = {stage3_out[7:0], 8'b00000000};
      end
      2'b01: begin    // Shift right
         stage1_in = {1'b0, stage0[15:1]};
         stage2_in = {2'b00, stage1_out[15:2]};
         stage3_in = {4'b0000, stage2_out[15:4]};
         stage4_in = {8'b00000000, stage3_out[15:8]};
      end
      2'b10: begin    // Rotate left
         stage1_in = {stage0[14:0], stage0[15]};
         stage2_in = {stage1_out[13:0], stage1_out[15:14]};
         stage3_in = {stage2_out[11:0], stage2_out[15:12]};
         stage4_in = {stage3_out[7:0], stage3_out[15:8]};
      end
      2'b11: begin    // Rotate right
         stage1_in = {stage0[0], stage0[15:1]};
         stage2_in = {stage1_out[1:0], stage1_out[15:2]};
         stage3_in = {stage2_out[3:0], stage2_out[15:4]};
         stage4_in = {stage3_out[7:0], stage3_out[15:8]};
      end
   endcase
end

generate
   for (i = 0; i < 16; i = i + 1) begin
      mux2x1 mux_stage1(
                        .a(stage0[i]),
                        .b(stage1_in[i]),
                        .sel(shamt[0]),
                        .y(stage1_out[i])
                       );
   end

   for (i = 0; i < 16; i = i + 1) begin
      mux2x1 mux_stage2(
                        .a(stage1_out[i]),
                        .b(stage2_in[i]),
                        .sel(shamt[1]),
                        .y(stage2_out[i])
                       );
   end

   for (i = 0; i < 16; i = i + 1) begin
```

```
        mux2x1 mux_stage3(
                        .a(stage2_out[i]),
                        .b(stage3_in[i]),
                        .sel(shamt[2]),
                        .y(stage3_out[i])
                    );
    end

    for (i = 0; i < 16; i = i + 1) begin
        mux2x1 mux_stage4(
                        .a(stage3_out[i]),
                        .b(stage4_in[i]),
                        .sel(shamt[3]),
                        .y(stage4_out[i])
                    );
    end
  endgenerate

  assign data_out = stage4_out;

endmodule
```

Description:

This module uses a log-stage barrel shifter to operate on a 16-bit input. At 4 stages and 16 muxes per stage, this design uses 64 total muxes generated by LUTS. The operation is selected by inputs to change direction and rotation, while the input "shamt" determines what stages of the shifter the input will go through

shamt_counter.v:

```
module shamt_counter(
                    input wire clk_1kHz,
                    input wire btn_raw,
                    output reg [1:0] shamt_high
                );

  reg [2:0] btn_val;
  reg btn_clean;
  reg btn_prev;

  initial begin
    btn_val = 3'b000;
    btn_clean = 1'b0;
    btn_prev = 1'b0;
    shamt_high = 2'b00;
  end

  always @(posedge clk_1kHz) begin
    btn_prev = btn_clean;
    btn_val = {btn_val[1:0], btn_raw};

    if(btn_val == 3'b111) btn_clean = 1;
    else if(btn_val == 3'b000) btn_clean = 0;
```

```
    if(btn_clean && !btn_prev)
        shamt_high = shamt_high + 1'b1;
  end

endmodule
```

Description:

Shamt counter uses a 2-bit free counter that advances on a button press. In order to circumvent the dual purpose asychronous reset the button is also used for, this module replicates the debounce module internally

hex_to_7seg.v:

```
module hex_to_7seg(
                    input [3:0] hex,
                    output reg [6:0] digit
                );

  always @(*) begin
    case(hex)
      4'd0: digit=7'b0000001;
      4'd1: digit=7'b1001111;
      4'd2: digit=7'b0010010;
      4'd3: digit=7'b0000110;
      4'd4: digit=7'b1001100;
      4'd5: digit=7'b0100100;
      4'd6: digit=7'b0100000;
      4'd7: digit=7'b0001111;
      4'd8: digit=7'b0000000;
      4'd9: digit=7'b0001100;
      4'd10:digit=7'b0001000;
      4'd11:digit=7'b1100000;
      4'd12:digit=7'b0110001;
      4'd13:digit=7'b1000010;
      4'd14:digit=7'b0110000;
      4'd15:digit=7'b0111000;
      default: digit=7'b1111111;
    endcase
  end

endmodule
```

Description:

This module is a simple hex lookup table for the active low 7seg display

seg7_scan8.v:

```
module seg7_scan8(
```

```
            input clk_1kHz,
            input rst_n,
            input [15:0] word,
            output reg [7:0] an,
            output reg [6:0] seg
        );

reg [2:0] sel;

wire [6:0] digit0;
wire [6:0] digit1;
wire [6:0] digit2;
wire [6:0] digit3;

hex_to_7seg hex0(
    .hex(word[3:0]),
    .digit(digit0)
);

hex_to_7seg hex1(
    .hex(word[7:4]),
    .digit(digit1)
);

hex_to_7seg hex2(
    .hex(word[11:8]),
    .digit(digit2)
);

hex_to_7seg hex3(
    .hex(word[15:12]),
    .digit(digit3)
);

always @(posedge clk_1kHz or negedge rst_n) begin
    if(!rst_n)
        sel <= 3'b000;
    else
        sel <= sel + 1'b1;
end

always @(*) begin
    case(sel)
        3'd0: begin
            an = 8'b11111110;
            seg = digit0;
        end
        3'd1: begin
            an = 8'b11111101;
            seg = digit1;
        end
        3'd2: begin
            an = 8'b11111011;
            seg = digit2;
        end
        3'd3: begin
            an = 8'b11110111;
            seg = digit3;
        end
        default: begin
            an = 8'b11111111;
```

```
            seg = 7'b1111111;
        end
    endcase
end

endmodule
```

Description:

This scanner uses the hex_to_7seg module four times in order to determine each digit it will display to the 7seg. On each 1 kHz clock cycle a different display is driven with its corresponding digit, including the unused upper displays that are turned off.

top_lab7.v:

```
module top_lab7(
                input CLK_100MHz,
                input [15:0] SW,
                input BTNC, BTNU, BTND, BTNL, BTNR,
                output wire [7:0] AN,
                output wire [6:0] SEG,
                output wire [7:0] LED
            );

    wire RST_N;
    assign RST_N = ~BTNC;

    wire CLK_1kHz;
    wire CLK_2Hz;

    wire DIR;
    wire ROTATE;

    wire [1:0] SHAMT_LOW;
    wire [1:0] SHAMT_HIGH;

    wire [15:0] BARREL_OUT;
    wire [15:0] RESULT_WORD;


    clock_divider_fixed clk_divs(
                            .clk_100MHz(CLK_100MHz),
                            .clk_2Hz(CLK_2Hz),
                            .clk_1kHz(CLK_1kHz)
                        );

    debounce_toggle debounce_btnu(
                                . btn_raw(BTNU),
                                .clk_1kHz(CLK_1kHz),
                                .rst_n(RST_N),
                                .btn_toggle(DIR)
                            );

    debounce_toggle debounce_btnd(
                                . btn_raw(BTND),
```

```
                .clk_1kHz(CLK_1kHz),
                .rst_n(RST_N),
                .btn_toggle(ROTATE)
            );

    debounce_toggle debounce_btnl(
                . btn_raw(BTNL),
                .clk_1kHz(CLK_1kHz),
                .rst_n(RST_N),
                .btn_toggle(SHAMT_LOW[1])
            );

    debounce_toggle debounce_btnr(
                . btn_raw(BTNR),
                .clk_1kHz(CLK_1kHz),
                .rst_n(RST_N),
                .btn_toggle(SHAMT_LOW[0])
            );

    shamt_counter shamt_count(
                  .clk_1kHz(CLK_1kHz),
                  .btn_raw(BTNC),
                  .shamt_high(SHAMT_HIGH)
              );

    barrel_shifter16 shifter(
                  .data_in(SW),
                  .shamt({SHAMT_HIGH, SHAMT_LOW}),
                  .dir(DIR),
                  .rotate(ROTATE),
                  .data_out(BARREL_OUT)
              );

    assign RESULT_WORD = BARREL_OUT;

    seg7_scan8 display(
                .clk_1kHz(CLK_1kHz),
                .rst_n(RST_N),
                .word(RESULT_WORD),
                .an(AN),
                .seg(SEG)
            );


    assign LED = {DIR, ROTATE, 2'b00, {SHAMT_HIGH, SHAMT_LOW}};

endmodule
```

Description:

The top level module instantiates and wires each module together as well as displaying LED readouts.

XDC Snippet:

```
# Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { CLK_100MHz }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK_100MHz}];


##Switches

set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }];
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }];
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }];
…
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }];
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }];
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }];

## LEDs

set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { LED[0] }];
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { LED[1] }];
…
set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];


#7 segment display

set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports {  SEG[6] }];
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports {  SEG[5] }];
…
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports {  SEG[1] }];
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports {  SEG[0] }];

set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { AN[0] }];
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { AN[1] }];
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { AN[2] }];
…
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { AN[5] }];
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { AN[6] }];
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { AN[7] }];


#Buttons

set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { BTNC }]; #IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { BTNU }]; #IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { BTNL }]; #IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { BTNR }]; #IO_L10N_T1_D15_14 Sch=btnr
set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }]; #IO_L9N_T1_DQS_D13_14 Sch=btnd
```

Simulation:

seg7_scan8_TB.v:

```
module seg7_scan8_TB(

                );

    reg clk_tb;
    reg rst_n_tb;

    reg [15:0] word_tb;

    wire [7:0] an_tb;
    wire [6:0] seg_tb;

    reg [7:0] expected_an;
    reg [6:0] expected_seg;

    integer i;
    integer tasksfailed;

    seg7_scan8 scan_tb(
                    .clk_1kHz(clk_tb),
                    .rst_n(rst_n_tb),
                    .word(word_tb),
                    .an(an_tb),
                    .seg(seg_tb)
                );

    initial
    begin
        clk_tb = 1'b0;
        rst_n_tb = 1'b0;

        word_tb = 16'hDCBA;

        tasksfailed = 0;
    end

    always
    begin
        #5 clk_tb = ~clk_tb;
    end

    initial
    begin
        @(posedge clk_tb);
        #1;
        rst_n_tb = 1'b1;

        for(i = 0; i < 8; i = i + 1) begin
            case(i)
                0: begin
                    expected_an = 8'b11111110;
                    expected_seg = 7'b0001000;
                end
```

```
            1: begin
               expected_an = 8'b11111101;
               expected_seg = 7'b1100000;
            end

            2: begin
               expected_an = 8'b11111011;
               expected_seg = 7'b0110001;
            end

            3: begin
               expected_an = 8'b11110111;
               expected_seg = 7'b1000010;
            end

            default: begin
               expected_an = 8'b11111111;
               expected_seg = 7'b1111111;
            end
         endcase

         if((an_tb !== expected_an) || (seg_tb !== expected_seg))
         begin
            tasksfailed = tasksfailed + 1;
            $display("FAILED: Digit = %d:    AN = %b, expected = %b    SEG = %b, expected = %b", i, an_tb, expected_an, seg_tb,
expected_seg);
         end

         if(i < 7) begin
            @(posedge clk_tb)
            #1;
         end
      end

      if(tasksfailed == 0)
         $display("All tests passed");
      else
         $display("%d tests failed", tasksfailed);

      $finish;
   end
endmodule
```

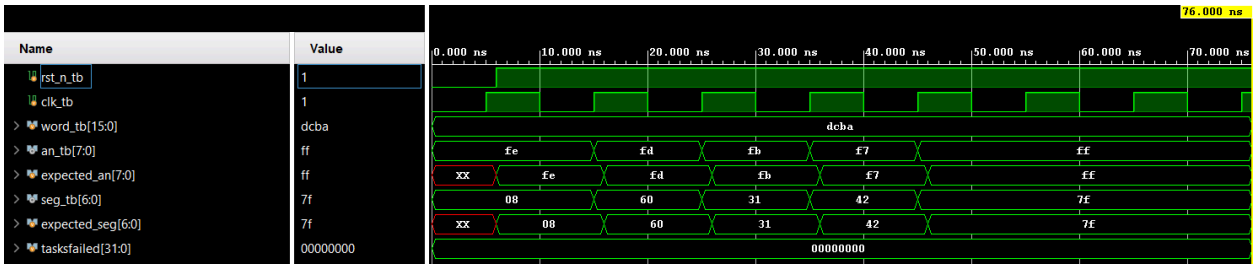
Description:

To check 1-of-8 enable sequence and segment codes, I input a distinct word "DCBA" and used automatic checks to verify the correct digit and display was active each clock cycle.

Output:

```
All tests passed
$finish called at time : 76 ns : File "D:/School/ECE3300/ECE3300 Lab7 GroupU
```

Waveform:



debounce_toggle_TB.v:

```
module debounce_toggle_TB(

                    );

    reg clk_tb;
    reg btn_raw_tb;
    reg rst_n_tb;
    wire btn_toggle_tb;

    debounce_toggle db_tb(
                .clk_1kHz(clk_tb),
                .btn_raw(btn_raw_tb),
                .rst_n(rst_n_tb),
                .btn_toggle(btn_toggle_tb)
               );

    initial
       begin
          clk_tb = 1'b0;
          rst_n_tb = 1'b0;
       end

    always
       begin
          #5 clk_tb = ~clk_tb;
        end

     initial
       begin
          @(posedge clk_tb)
          #1;
          rst_n_tb = 1'b1;

          btn_raw_tb = 1'b0;              // Test 1: Stable low
          repeat(6) @(posedge clk_tb);
          #1;

          if(btn_toggle_tb !== 1'b0)
          begin
             $display("FAIL: btn_toggle should be 0 after stable low");
             $fatal;
          end
```

```
        btn_raw_tb = 1'b1;              // Test 2: Bounce
        @(posedge clk_tb);
        #1;

        btn_raw_tb = 1'b0;
        @(posedge clk_tb);
        #1;

        if(btn_toggle_tb !== 1'b0)
        begin
           $display("FAIL: btn_toggle changed after bounce");
           $fatal;
        end

        btn_raw_tb = 1'b1;             // Test 3: Toggle on
        repeat(6) @(posedge clk_tb);
        #1;
        btn_raw_tb = 1'b0;
        repeat(6) @(posedge clk_tb);

        if(btn_toggle_tb !== 1'b1)
        begin
           $display("FAIL: btn_toggle should be 1 after stable high");
           $fatal;
        end

        btn_raw_tb = 1'b1;              // Test 4: Bounce
        @(posedge clk_tb);
        #1;

        btn_raw_tb = 1'b0;
        @(posedge clk_tb);
        #1;

        if(btn_toggle_tb !== 1'b1)
        begin
           $display("FAIL: btn_toggle changed after bounce");
           $fatal;
        end

        btn_raw_tb = 1'b1;             // Test 3: Toggle off
        repeat(6) @(posedge clk_tb);
        #1;
        btn_raw_tb = 1'b0;
        repeat(6) @(posedge clk_tb);

        if(btn_toggle_tb !== 1'b0)
        begin
           $display("FAIL: btn_toggle should be 0 after stable high");
           $fatal;
        end

        $display("Passed all tests");
        $finish;
    end


endmodule
```

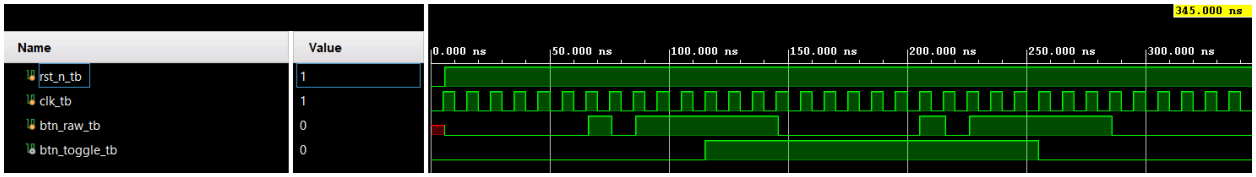
Description:

To verify the debounce and toggle implementation we tested random bounces and automatically verified that they were filtered. We also tested that toggle only switches on active high

Output:

```
Passed all tests
$finish called at time : 345 ns : File "D:/School/ECE3300/ECE3300 Lab7 GroupU/ECE3300 Lab7 GroupU
```

Waveform:



barrel_shifter16_TB.v:

```
module barrel_shifter16_TB( );

    wire [15:0] result;
    reg [15:0] check_val;
    reg [15:0] in_val;
    reg [3:0] shamt;
    reg dir, rot;

    barrel_shifter16 barrel_TB(.data_in(in_val),
    .shamt(shamt), .dir(dir), .rotate(rot), .data_out(result));

    integer i, j, tasksfailed;
    initial begin
        in_val = 16'b0;
        shamt = 4'b0;
        check_val = 0;
        i = 0;
        tasksfailed = 0;
        //Rotate Right
        dir = 1;
        rot = 1;
        in_val = 16'hFAC3;
        check_val = {in_val[14:0], in_val[15]};
        for(j = 0; j < 16; j = j + 1)begin
            shamt = j;
            check_val = {check_val[0], check_val[15:1]};
            #10;
            if(result != check_val)begin
                tasksfailed = tasksfailed + 1;
```

```
            $display("Test Failed at %d", j);
        end
    end

    //Rotate Left
    dir = 0;
    rot = 1;
    in_val = 16'hFAC3;
    check_val = {in_val[0], in_val[15:1]};
    for(j = 0; j < 16; j = j + 1)begin
        shamt = j;
        check_val = {check_val[14:0], check_val[15]};
        #10;
        if(result != check_val)begin
            tasksfailed = tasksfailed + 1;
            $display("Test Failed at %d", j);
        end
    end

    //Logical Shift Right
    dir = 1;
    rot = 0;
    in_val = 16'h8000;
    check_val = 16'h0001;
    for(j = 0; j < 16; j = j + 1)begin
        shamt = j;
        check_val = {check_val[0], check_val[15:1]};
        #10;
        if(result != check_val)begin
            tasksfailed = tasksfailed + 1;
            $display("Test Failed at %d", j);
        end
    end

    //Logical Shift Left
    dir = 0;
    rot = 0;
    in_val = 16'b1;
    check_val = 16'h8000;
    for(j = 0; j < 16; j = j + 1)begin
        shamt = j;
        check_val = {check_val[14:0], check_val[15]};
        #10;
        if(result != check_val)begin
            tasksfailed = tasksfailed + 1;
            $display("Test Failed at %d", j);
        end
    end
    $display("%d Tests failed", tasksfailed);
    $finish;
  end

endmodule
```

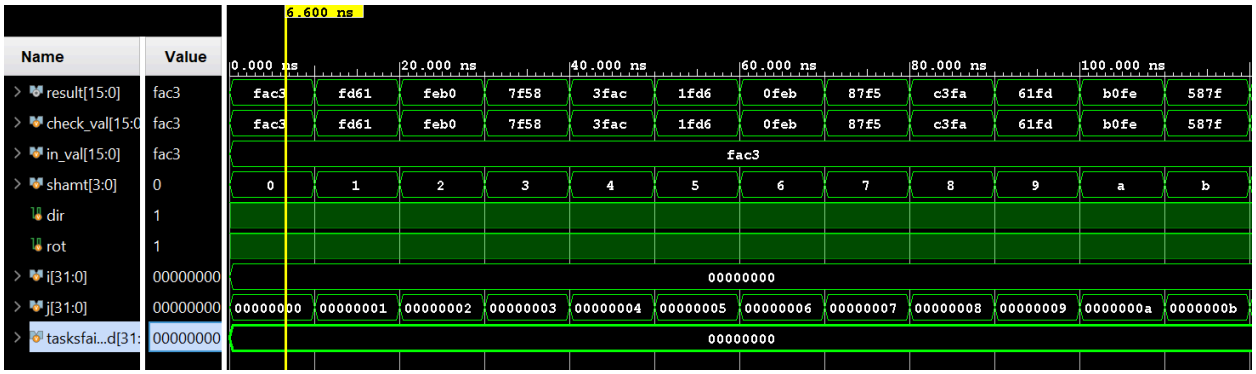
Description:

In this test bench the barrel shifter module is tested with an input, FAC3, which generates unique outputs for any size of shift in the rotate setting. This input is shifted through all sixteen bits in both directions. To test the logical shift a single bit is set and shifted through all sixteen bit positions, demonstrating the functionality of the barrel shifter.

Output:

```
Time resolution is 1 ps
          0 Tests failed
$finish called at time : 640 ns : File "C:/Users/rbSte/Documents/ECE_3300/ECE_3300_Lab/ECE_3300_Lab/ECE3300-Lab7-GroupU/Simulation/barrel
```

Waveform:



Implementation:

Resource Utilization Report:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	Slice (15850)	LUT as Logic (63400)	Bonded IOB (210)	BUFGCTRL (32)
top_lab7	113	45	7	44	113	45	1

Timing Report:

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.865 ns	Worst Hold Slack (WHS):	0.265 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	33	Total Number of Endpoints:	33	Total Number of Endpoints:	18

All user specified timing constraints are met.

Group Video:

https://youtu.be/esJ6hO58ZkA

Contributions:

Robert Stevenson: Core design, testbenches
Ben Robles: Design/Implementation, testbenches

Reflections:

Ben Robles: I enjoyed tackling more complex combinational logic and RTL design in this lab as well as continuing to build off the modules we've been working on. This lab was harder than the last, but not too bad at all.

Robert Stevenson: I enjoyed this lab, it was interesting and enjoyable to work on these complex combinational circuits. Although little of my original design for the barrel shifter made it to the final design, it was an enjoyable experience to overcome the challenge of designing a functional, if inelegant, barrel shifter.